Prescripto AI 2.0 — Deployment & Execution Dossier

Document ID: `DOC-11-DEP`
Classification: Academic Engineering Dossier
Canonical Spec Reference: Dockerfile, render.yaml, docker-compose.yml, docs/RENDER_DEPLOYMENT_GUIDE.md
Date of Extraction: November 4, 2026

---

1. Hosting Architecture Overview

Prescripto AI 2.0 is designed for hybrid multi-cloud deployment:

```
┌──────────────────────────────────────────────────────┐
│                   VERCEL CLOUD                       │
│  - Hosts: React 19 / Vite Single-Page Application    │
│  - Build Command: npm run build                      │
│  - Output Directory: dist/                           │
│  - SPA Routing: vercel.json (/* -> /index.html)      │
└──────────────────────────┬───────────────────────────┘
                           │ HTTPS REST Calls
┌──────────────────────────▼───────────────────────────┐
│                   RENDER CLOUD                       │
│                                                      │
│  1. Web Service (prescripto-2-0)                     │
│     - Docker Multi-Stage Container                   │
│     - Uvicorn ASGI Server on Port 10000              │
│     - Live URL: https://prescripto-2-0.onrender.com  │
│                                                      │
│  2. Managed Relational Database (prescripto-db)      │
│     - PostgreSQL 16 Managed Server                   │
│     - Connected via Internal Connection String       │
└──────────────────────────────────────────────────────┘
```

---

2. Multi-Stage Containerization (`Dockerfile`)

- **Source File:** Dockerfile

The root Dockerfile employs a 2-stage build pipeline:

```
# Stage 1: Build Frontend Single Page Application
FROM node:20-alpine AS frontend-builder
WORKDIR /web
COPY web/package.json web/package-lock.json* ./
RUN npm install
COPY web/ ./
RUN npm run build

# Stage 2: Python Backend Runtime
FROM python:3.11-slim

ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    PIP_NO_CACHE_DIR=1 \
    PYTHONPATH=/app/backend:/app

WORKDIR /app

# System dependencies for psycopg2, image libraries, and health probes
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    libpq-dev \
    curl \
    libgomp1 \
    && rm -rf /var/lib/apt/lists/*

COPY backend/requirements.txt ./backend/
RUN pip install --upgrade pip && pip install -r backend/requirements.txt

COPY backend/ ./backend/
COPY --from=frontend-builder /web/dist ./web/dist
COPY render.yaml ./

WORKDIR /app/backend

ENV PORT=8000
EXPOSE 8000

CMD ["sh", "-c", "alembic upgrade head && python scripts/seed_medications.py && python scripts/s
```

Key Engineering Features of the Container

1. **Self-Contained Unified Deployment Option**: Stage 2 copies the compiled React SPA from `frontend-builder` into `/app/web/dist`. This enables the backend container to serve both the REST API and the React frontend directly from a single service on `https://prescripto-2-0.onrender.com/`.
2. **Reverse Proxy SSL Header Preservation**: The `--proxy-headers --forwarded-allow-ips='*'` flags tell Uvicorn to trust incoming `X-Forwarded-Proto` and `X-Forwarded-For` headers from Render's cloud load balancers. This prevents mixed-content HTTPS blocks on Swagger UI (`/docs`).

3. **Idempotent Bootstrapping**: On every container launch, the startup command runs database migrations (`alembic upgrade head`), seeds verified medications (`seed_medications.py`), seeds demo staff/doctor accounts (`seed_users.py`), and seeds the admin account (`create_user.py`).

---

3. Environment Variables Reference

Environment Variable Name	Deployment Target	Required?	Purpose / Recommended Value
`DATABASE_URL`	Render Web Service	**Yes**	PostgreSQL connection URL (e.g. `postgresql://prescripto:...@dpg-...-a/prescripto_db`) or `sqlite:///prescripto.db`
`ENVIRONMENT`	Render Web Service	Yes	Runtime mode: `production` or `development`
`LOG_LEVEL`	Render Web Service	Yes	Structured logging verbosity: `INFO` or `DEBUG`
`CORS_ORIGINS`	Render Web Service	Yes	Allowed origins: `*` (or explicit Vercel domain)
`RECOGNITION_PROVIDER`	Render Web Service	Yes	Primary perception engine: `gemini` or `rapidocr`
`GEMINI_MODEL`	Render Web Service	Yes	Primary vision model: `gemini-3.5-flash`
`GEMINI_FALLBACK_MODEL`	Render Web Service	Yes	Secondary failover model: `gemini-3.1-flash-lite`
`GEMINI_API_KEY`	Render Web Service	Yes	API token for Google GenAI multimodal perception
`JWT_PRIVATE_KEY`	Render Web Service	Optional	2048-bit RSA PKCS#8 private key (auto-generated if omitted)
`JWT_PUBLIC_KEY`	Render Web Service	Optional	2048-bit RSA public key (auto-generated if omitted)
`PORT`	Render Web Service	Auto	Assigned dynamically by Render (defaults to `10000`)
`VITE_API_BASE_URL`	Vercel Frontend	**Yes**	Target backend API root: `https://prescripto-2-0.onrender.com`

---

4. Local Development Setup Guide

For a new software engineer setting up the project locally from scratch:

Prerequisites

- Python 3.11 or 3.12
- Node.js 20+ and npm

- Docker Desktop (optional, for MinIO and PostgreSQL)

Step 1: Clone & Configure Backend Environment

```
# 1. Clone repository
git clone https://github.com/dipak0000812/Prescripto-2.0.git
cd Prescripto-2.0

# 2. Set up Python virtual environment
python -m venv .venv
# Windows PowerShell:
.venv\Scripts\Activate.ps1
# Linux/macOS:
source .venv/bin/activate

# 3. Install backend dependencies
python -m pip install --upgrade pip
pip install -r backend/requirements.txt

# 4. Create local environment file
Copy-Item backend\.env.example backend\.env
```

Step 2: Initialize Database & Seed Master Data

```
cd backend

# Run database migrations
alembic upgrade head

# Seed 107 master medications into the formulary
python scripts/seed_medications.py

# Seed demo clinician and admin accounts
python scripts/seed_users.py
python scripts/create_user.py
```

Step 3: Run the Backend Server

```
# From Prescripto-2.0/backend:
uvicorn prescripto.api.main:app --host 127.0.0.1 --port 8000 --reload
```

- Interactive API Documentation (Swagger UI): `http://127.0.0.1:8000/docs`
- System Health Diagnostic Endpoint: `http://127.0.0.1:8000/health`

Step 4: Run the Frontend Application

```
# Open a new terminal from Prescripto-2.0/web:
cd web
npm install
npm run dev
```

- Frontend Local Development URL: `http://localhost:5173`
- Pre-configured Demo Doctor Login:
- **Email:** `doctor.sharma@prescripto.local`
- **Password:** `DoctorPass123!`
- Pre-configured Patient Login:
- **Phone Number:** `9876543210`
- **OTP:** `123456`